

Routing Engine — User Flow Walkthrough

How the routing engine works from the moment a user searches for a route.

Phase 1 — App Startup (Before You Even Search)

Before any user opens the app, the server pre-builds everything.

Graph Construction (`src/routing/road_graph.py`)

The Toronto Centreline GeoJSON is loaded into a `networkx.DiGraph`. Each road segment becomes a directed edge between two intersection node IDs (`FROM_INTERSECTION_ID` → `TO_INTERSECTION_ID`). One-way streets get a single edge; bidirectional streets get two edges (forward + reverse). Invalid rows (null IDs, degenerate geometries) are skipped silently.

Node Geometry (`src/routing/road_graph.py`)

Nodes are raw numeric IDs — no coordinates. A separate pass (`build_node_geometry()`) re-projects everything to EPSG:4326 and assigns each node its lat/lon by reading the first/last coordinate of whichever segment it appears in first.

Travel Time Estimation (`src/routing/road_graph.py`)

At graph-build time, every edge gets a `travel_time_hours` attribute using fixed speed assumptions — no live traffic data.

Road Class	Speed
Arterial	50 km/h
Minor Arterial	45 km/h
Collector	40 km/h
Local	30 km/h
Unknown	40 km/h

Formula:

```
travel_time_hours = segment_length_m / (speed_kmh / 3.6) / 3600
```

ML λ Map (`backend-api/app.py`)

The Hurdle-Temporal model runs `predict_lambda()` and produces a `segment_id → λ (crashes/hour)` map for every road segment. This is stored in memory and reused per request.

Phase 2 — You Type a Destination and Hit "Go"

Your origin + destination arrive at `/api/routes/safety-aware` as lat/lon pairs, plus optional `weather` and `time_of_day` context.

Snap to Graph

Both points are snapped to their nearest graph node within **300 meters** using a flat-earth approximation (`degrees × 111,000`). If nothing is within 300 m, the request fails. This converts GPS coordinates into the node IDs Dijkstra can work with.

Risk Multipliers Applied (`backend-api/app.py`)

Before routing, the λ values are scaled by a combined multiplier:

```
combined_mult = weather_mult × time_mult
λ_adjusted    = λ × combined_mult    (for every segment)
```

Weather multipliers:

Condition	Multiplier
Snow / Thunderstorm	1.50×
Sleet	1.40×
Rain	1.35×
Fog / Mist	1.20×
Clear	1.00×

Time-of-day multipliers:

Period	Multiplier
Late night (23:00–05:00)	1.30×
Rush hour — weekday	1.25×
Rush hour — weekend	1.10×
All other hours	1.00×

Phase 3 — Two Routes Are Computed

Risk Weighting Applied (`src/routing/road_graph.py`)

Each edge gets a second weight derived from its adjusted λ :

```
expected_crashes =  $\lambda_{\text{adjusted}}$  * travel_time_hours  
risk_weight_hours = travel_time_hours +  $\beta$  * expected_crashes
```

$\beta = 0.1$ hours/expected crash (tunable per request via the `beta` POST body field). This converts crash risk into a time penalty — making a risky segment appear "slower" to the optimizer.

Dijkstra × 2

Route	Weight minimized
Fastest route	travel_time_hours
Safer route	risk_weight_hours

Phase 4 — Risk Score Computed for Each Route

For each route, risk is aggregated across all edges using a Poisson model:

```
 $\Lambda = \sum (\lambda_i \times t_i)$  ← total expected crashes across all edges  
 $P = 1 - e^{-\Lambda}$  ← probability of at least one crash (Poisson CDF)
```

The API response includes both routes with:

- `expectedCrashes`
- `routeProbability`
- `totalTravelTimeHours`

Phase 5 — Map Display

Route Geometry (`src/routing/maps_export.py`)

Per-edge `LineString` geometries are stitched together, Douglas-Peucker simplified (`tolerance=0.`

$0.001^\circ \approx 10$ m), capped at 25 waypoints, and returned as (lat, lon) pairs.

Risk Color Labels

Road segments on the map are colored low / medium / high based on where their λ falls relative to percentile thresholds computed once at startup:

Label	Threshold
Low	$\lambda \leq p70$
Medium	$p70 < \lambda \leq p90$
High	$\lambda > p90$

These thresholds are derived from the full distribution of λ across all segments — so risk labels are always relative to the city-wide baseline.